

Terraform Drift Detection: Catch Hidden Changes Fast

Cloud drift happens when someone manually changes settings in your cloud portal outside of code. Learning how to detect and fix drift prevents broken production deployments and keeps your infrastructure secure.

What you'll learn:

- 1 Catching Drift in CI/CD Pipelines
- 2 Updating State Safely with Refresh-Only
- 3 Recreating Corrupted Resources Cleanly
- 4 Importing Untracked Portal Resources

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Catching Drift in CI/CD Pipelines

Automation pipelines need a reliable way to check if real cloud infrastructure matches your Terraform code. By using the detailed exit code flag, Terraform returns exit code 0 for no changes, exit code 2 if drift is found, and exit code 1 on errors. This allows your build pipelines to block bad deployments automatically.

```
$terraform plan -detailed-exitcode
```

Cloud & DevOps Daily

Terraform Drift Detection: Catch Changes Before They Break Prod

Wednesday, September 16, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Updating State Safely with Refresh-Only

When infrastructure changes intentionally outside Terraform, your state file becomes outdated. Running a refresh-only apply updates your local state file with current cloud settings without making any changes to live resources. This keeps state accurate without risking unwanted downtime.

```
$ terraform apply -refresh-only
```

Cloud & DevOps Daily

Terraform Drift Detection: Catch Changes Before They Break Prod

Wednesday, September 16, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Recreating Corrupted Resources Cleanly

If manual changes break a cloud resource completely, trying to patch it can take hours. The replace flag tells Terraform to mark a specific resource for destruction and recreation during the next deployment. This restores your resource to a known good state defined by code.

```
$ terraform apply -replace="aws_instance.web_server"
```

Cloud & DevOps Daily

Terraform Drift Detection: Catch Changes Before They Break Prod

Wednesday, September 16, 2026

Swipe to tip 4 >> linkedin.com/in/mrmanrajsingh

4

TIP 4 OF 4

Importing Untracked Portal Resources

If a colleague created a database or storage bucket directly in the cloud console, you don't need to rebuild it. The import command links the live cloud resource ID to your Terraform code block. Once imported, Terraform tracks future changes cleanly.

```
$terraform import aws_s3_bucket.data my-app-data-bucket
```

Cloud & DevOps Daily

Terraform Drift Detection: Catch Changes Before They Break Prod

Wednesday, September 16, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Schedule daily automated drift detection scans in your CI/CD tool.
- + Restrict manual edit permissions in cloud management portals.
- + Always review terraform plan output carefully before running apply.
- + Use remote state storage with state locking enabled to prevent conflicts.
- + Keep Terraform modules small so state scans remain quick and responsive.

Watch Out For

- ! Running terraform apply without reading the plan summary first.
- ! Editing state files manually instead of using terraform state commands.
- ! Forgetting to write matching resource code before running terraform import.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh